

Markdown is the substrate

A manifesto for the live-document runtime

Christian Beaumont, Foundation42

2026-05-16

Markdown is not a presentation format. *It is the working medium of humans, agents, and machines together. Treat it as such. Build the runtime that makes it live.*

This is what we are building in `~/dev/terminal/`. A live-document runtime where markdown is the **declarative interface** to a component-driven, Vulkan-native, LLM-aware substrate. Not a viewer. Not a renderer-on-the-side. The medium itself.


text_engine_demo
⌵ ⌶ ✕

Live components

Block extensions parse into Specs, get resolved through a host-owned registry into cached instances, and survive across re-parses. Templated `${state.x}` attrs subscribe to state mutations through the reactive layer. The host's frame loop streams `:::update {target=state.box_color}` directives at 1.5s intervals — watch the box recolour through the same fast lane an LLM would use.



Flex layout (stage 15c)

The first multi-child constraint-aware provider. `:::flex` walks its children with cumulative-x positioning, dropping gap pixels between siblings. Each child still goes through the kiwi solver for its own bounds — the flex parent computes positions, the children constrain their sizes.



Grid layout (stage 15d)

The second multi-child constraint-aware provider. `:::grid` lays its body out into equal-width tracks (`columns=N`), row-major, gap between cells, auto-height rows sized to the tallest cell. Each cell still goes through the kiwi solver for its own bounds — the grid parent allocates the track, the cell constrains its size inside it. `:::flex` handles 1D; `:::grid` handles 2D; both stack cleanly inside each other.



Composition (stage 9)

The block below is a *whole other document* — `src/widgets/orbit_panel.md` — embedded recursively. Parent attrs (`panel_color=cyan, inner_color=magenta`) overlay the child's frontmatter; child components live in the same registry under the scope `orbit/...` so their ids can't collide with the parent's.

A nested document. Frontmatter values become the **child state**; parent attrs on the embed overlay onto them at create time so the embed line in `demo.md` wins on conflict.

The runtime mid-flight — live components, `:::flex` arranging a row of boxes, `:::grid` composing a six-cell dashboard, an embedded sub-document grafted in below — all from markdown source, all in one Vulkan pass.

Originally written 2026-05-16, mid-session-9, in response to the recent push from some quarters to default LLM outputs to HTML instead of markdown. We disagree, and the

disagreement is structural — not aesthetic. Last updated 2026-05-16, end of session 13 — homoiconic claim sharpened through external review. The middle has settled; the claim still stands.

The wrong fork

The argument is being made that LLMs should default to HTML output: richer presentation, denser information, interactive artifacts. With a million-token context window, the token overhead is “negligible”. With a browser as the viewer, the rendering is “free”.

This is the view from inside a closed ecosystem. From outside it, the calculus inverts.

- **Markdown diffs.** HTML diffs are a wall of tags around a three-character textual change. Markdown is line-oriented and fits the way humans, version control, and code review actually work. The “free” presentation costs you everything downstream.
- **Markdown round-trips.** Markdown is something a human can read in a terminal, paste into an issue tracker, commit to a repo, and an LLM can read back without parsing. HTML round-trips only inside a browser.
- **Tokens are budget.** A million-token context is not an excuse for inefficient encoding; it is a resource to spend on *reasoning*. Every `</td></tr><tr><td>` is a token the model could have spent on the answer.
- **Structural tracking is real cost.** Even frontier models drop closing tags under load. Smaller, edge-deployed, locally-run models hit the wall much sooner. HTML-first defaults pull the ladder up behind the only models that can afford the format.
- **Extension is the seam; capture is closing it.** Markdown gets extended — it has happened at least twice already (GFM, MDX), and our own `:::name {attrs}` is the third. The question isn’t whether to extend; it’s whether the extension layers onto a surface anyone can read or whether it locks rendering to a single viewer. HTML-first output looks rich inside a specific renderer and is sludge outside it. That isn’t progress; it’s a moat. The geology says extension wins on the open surface; capture is what loses.

CommonMark already won the format war for a reason. It is the ground that humans, agents, terminals, READMEs, PR descriptions, issue trackers, and `gpt-{anything}` all stand on. Anything richer should be **layered on top of markdown, not replace it**.

That is what this project is.

What we believe

1. Markdown is ground truth.

It is what flows between humans and agents in 2026, and it is what should flow inside our substrate too. The renderer's job is to make markdown more capable, not to replace it with a format only the renderer can read.

2. Extend, don't replace.

CommonMark has an extension point: fenced blocks with attributes. We use `:::name {attrs}` for block extensions. An HTML block in the middle of a markdown doc is the same shape. Mermaid, KaTeX, custom widgets — all of these layer in without leaving the format.

```
:::chart {#telemetry capacity=600 color=cyan}
:::

:::llm-stream {provider=openai model=deepseek-chat
               prompt="explain entropy in three lines"}
:::

:::svg-stream {model=recraft-v3 prompt="A bowl of petunias"}
:::
```

The renderer dispatches `:::name` to a registered factory. Everything else is markdown. A human can read the file. Git can diff the file. An LLM can author the file. The runtime makes the file *do things*.

3. Tokens are precious.

Every output token costs latency, money, and reasoning bandwidth. A format that makes the model emit `<span class="hl">x</span>` to say `**x**` is a regression dressed as progress. The substrate's job is to do more with fewer tokens, not the reverse.

4. Close to the metal.

Zig + Vulkan, no web layer, no DOM, no JavaScript runtime. Glyphs go through HarfBuzz to a custom atlas to one `vkCmdBeginRendering` pass. Quads, triangles, raster images, text — one pipeline order, one draw stream. ~7,600 fps Release at idle with the full demo loaded; ~13k fps when nothing is animating. Performance is not a feature — it is the *budget* we spend on the live, reactive, per-glyph behaviour that the LM tier needs.

5. Composition is a flywheel.

Documents *are* components. `:::embedded-document {src=...}` is a built-in factory that parses another markdown file (local, HTTP, content-hash, future MCP) and grafts it into the

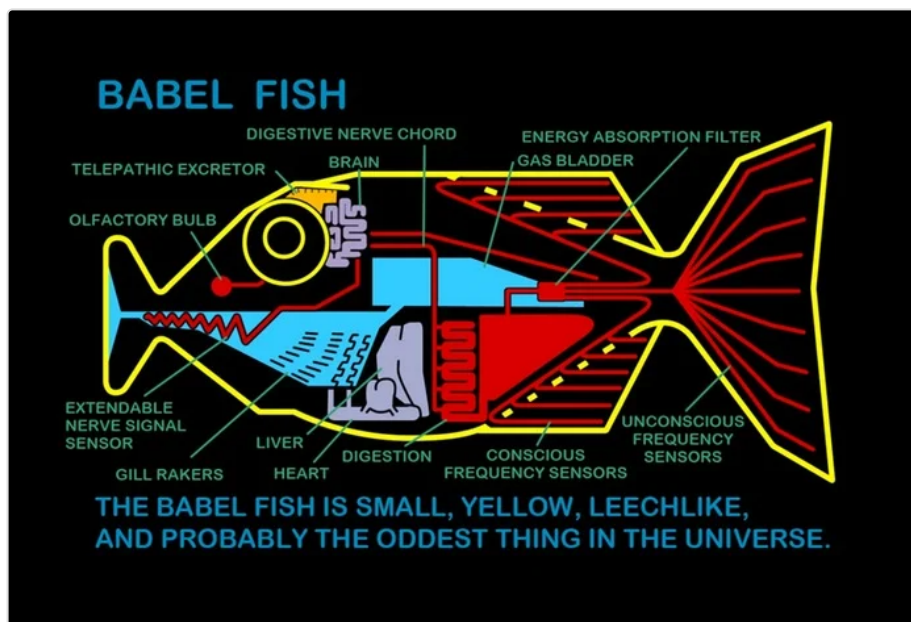
host. Reactive state crosses the network boundary. Headless documents run as pure state machines with no viewport. The friction to publish a new component is the friction of writing a markdown file. The substrate gets richer every time someone writes a document.

6. Provenance is a host policy.

A component can come from a built-in factory, a remote RPC, a WASM module, an LLM inference call, or a SPIR-V shader. The `Factory` contract (`create` / `update?` / `handle_update?` / `deinit?`) is deliberately provenance-agnostic. The same registry that holds `:::box` holds `:::llm-stream` holds `:::svg-stream`. Five provenances, one shape.

7. The LLM is a component author, not just a chat partner.

The LM tier streams markdown into the document. The document is live. `:::update {#id action=...}` directives skip layout entirely and route bytes to a named component's buffer at >10kHz. This is the loop: human writes markdown → markdown becomes a live runtime → LM streams more markdown into it → reactive state propagates → components mutate → screen updates. The same format the human authored, the LM amends. There is no impedance mismatch.



A Babel fish, generated live by `:::svg-stream` against Recraft V4.1 — vector graphics authored by the same LLM tier that authors the document around it. The SVG arrives, gets tessellated in parallel across the compute pool (8.17× speedup), and renders through the same triangle pipeline as everything else.

8. The substrate is homoiconic.

Homoiconicity is representational identity between program text and the data the language manipulates. It says nothing about determinism — `(random)` doesn't return the same value twice and nobody calls that a crack in LISP. The claim decomposes into four axes, all the same kind of property:

i. Representational identity. The serialised form (markdown text) and the runtime form (parsed component tree feeding the layout walker) are the same shape. An author types `:::grid`, the parser produces a Grid component, the renderer lays it out, and the result serialises back to `:::grid`. A `:::flex` inside a `:::grid` inside an `:::embedded-document` is the same tree to the runtime as it is to the eye reading the file. No XAML between author intent and runtime behaviour; no JSX compiler turning one syntax into a different tree; no DOM that is a different artifact from its source.

ii. Source/image gap closed in practice. A typical LISP image does not rewrite its own on-disk source as it runs — exotic, rare. Our substrate’s normal operating mode is that. The document tree IS the source representation; when `:::llm-stream` streams output, it streams markdown directly into the live document, and serialising the runtime hands back markdown the parser can re-eat. No lossy compile step between source and image.

iii. Modification surface in authored syntax. At the document layer there is no structural API by which a component reaches across the tree and rewrites siblings. Composition happens by emitting markdown the parser re-eats; no substructural mutation interface exists. The runtime underneath constructs element trees directly in Zig — that’s plumbing, same as LISP’s `read` is C code building cons cells — but the only mutation surface the document exposes to its own components, to LMs streaming into it, or to human authors, is the surface syntax. Tree changes happen by re-parse, not by structural poke. This is the load-bearing invariant: **runtime state must always be representable in source**. Any future API that breaks this would first have to argue the substrate’s central claim was wrong.

iv. Surface as fixed point. The serialised surface is simultaneously three things at once: the surface a human authors, the form the runtime executes, and the representation the comprehender (LM) is most fluent in. One artifact, three readers — human, runtime, model — no translation layer between any pair. This is substrate-intrinsic, not plugin-relative: bolt an LM onto a LISP REPL and you get comprehension, but the model’s high-fluency surface and S-expressions don’t coincide, so the impedance wall stays. Ours coincide. And the coincidence isn’t a 2026 corpus accident — markdown is the comprehender’s high-fluency surface because it is the human’s structured-prose surface, and any future comprehender useful to humans is selected for fluency in the human-legible surface. The triple coincidence is a convergent attractor, not a temporary fact. The HTML-first push isn’t a world where this coincidence is absent — it’s the choice not to stand on it. The fixed point holds; the ecosystem declines the surface anyway. That is what “The wrong fork” is about.

This is the LISP property in a UI context — and it generalises one axis further than LISP itself. Markdown is the LISP of collaborative documents: the runtime accepts the same shape it emits, an LM streams more of the same shape, a human reads the result. An LLM streaming markdown is streaming UI; a `:::input` targeting an `:::llm-stream` is a

document extending itself. HTML doesn't have this property; XAML can't reach it; React's fiber tree is a different artifact from its JSX source.

There is no compile step between writing UI and running UI. There is no separate format the renderer “understands” that the source doesn't. There is no template language standing between author and pixels. The format is the program; the program is the format; the parse is the build — and the surface is the fixed point where author, runtime, and comprehender meet without translation.

text_engine 🚀 ✨

Live-document runtime — colour emoji 🍷 flows inline with text 🌸, mathematical signs ✓
 $X \approx \neq \infty \leq \geq$ pick up the body font, and rich glyphs (rocket 🚀, sparkles ✨, flower 🌸,
palette 🍷, party 🎉, star ★, fire 🔥, heart ❤️) ride the colour atlas alongside
everything else.

Stage 3 — markdown source becomes the tree

Same renderer as before — only the construction path changed. This document is parsed
by `cmark` into an AST, then walked into an `element.Element` tree.

Inline cascade

This paragraph uses *italic* and **bold** and ***bold-italic*** and `inline code` and a [link](#).
Underlines honour wrap — [this is a longer link with enough text inside its anchor that it
spans more than one line, producing one underline quad per visible line of the link.](#)

Nesting

- block kinds nest
- indent shrinks `max_w` for nested content
- and items hold multiple blocks:
 1. first nested
 2. second nested

Quotes indent their content, and indent propagates through Constraints so the
inline-flow inside this quote wraps on the narrower available width — not on the full
viewport width.

Code

```
fn render(elem: Element) !void {
    // code blocks: monospace, preformatted
}
```

A ```ansi fence` (stage 5b) hands its body to the ANSI parser and embeds the result back
as `CodeContent.sub_block` — the layout walker recurses, so SGR foreground colours,
bold, *italic*, underline (4/24), strikethrough (9/29), and reverse video (7/27) all render
inside the same code-block chrome as raw fences:

```
text_engine ✓ build ok
error: out of fuel (recoverable)
loaded glyph_cache + atlas + shaper
truecolor supported via SGR 38;2;R;G;B
underline + strikethrough + reverse video
bold underline cyan, red-strike, yellow reverse
```

Live components

Block extensions parse into Specs, get resolved through a host-owned registry into cached
instances, and survive across re-parses. Templated `${state.x}` attrs subscribe to state
mutations through the reactive layer. The host's frame loop streams `:::update`
`{target=state.box_color}` directives at 1.5s intervals — watch the box recolour through
the same fast lane an LLM would use.

Markdown source becomes the runtime tree: inline cascade, nesting, code blocks, live components — all the same shape to the parser, the renderer, and the eye reading the file. The format is the program; the program is the format.

What's built

Thirteen sessions in (2026-05-14 → 2026-05-16). The substrate is real, not a sketch.

- **Element contract** — one tagged union (text, paragraph, heading, list, code, quote, custom, etc.) that *every* producer emits and *every* consumer walks. Open at the seams via `custom { vtable, ctx }`.
- **Markdown engine** — `cmark 0.31.2` vendored; full CommonMark plus `:::name {attrs}` block extensions, YAML frontmatter reactive state, `${state.x}` interpolation.
- **ANSI engine** — same Element contract, second producer. UTF-8
 - SGR + 256-colour + truecolor. Closes the markdown↔terminal composability loop.
- **Live components** — `:::box`, `:::chart`, `:::slider`, `:::button`, `:::input`, `:::svg`, `:::embedded-document`, `:::llm-stream`, `:::svg-stream`, `:::image-stream`.
- **Reactive state** — single global store, `${path}` interpolation, per-binding subscribers, dirty-bubble across embedded documents.
- **Async I/O** — work-stealing job pool for compute, dedicated blocking-I/O pool for HTTP and LLM streams. Main thread never blocks. LLM streams ride the same channel as file watchers, remote-document fetches, and image generation.
- **LLM authoring** — `:::llm-stream` against any `POST /chat/completions` SSE provider (DeepSeek, OpenRouter, OpenAI, Together, Groq, Mistral, local Ollama). Re-parses the growing message as markdown on every chunk; the document grows token-by-token, live.
- **Vector + raster authoring** — `:::svg-stream` (Recraft V4.1) generates SVGs from prompts; tessellated in parallel across the compute pool (8.17× speedup), drawn through a Vulkan triangle pipeline. `:::image-stream` (Gemini image preview) decodes base64 PNG into a per-component `VkImage`.
- **Recursive composition** — `:::embedded-document` over local paths or HTTP URLs. Scoped cache keys. Parent-overlay attrs. State dirty bubbles up; subscribers stay scoped.
- **Persistent layout cache** — retained per-block layout output, content-version keyed. 97.9% hit rate at idle. The chart can fire at 60 Hz without re-walking the paragraph above it. Parallel cache-miss walks fan out across a work-stealing pool when the cost threshold justifies it.
- **Constraint substrate** — pure-Zig port of Cassowary (the kiwi C++ solver), ~3,000 LOC and 300+ tests. Wrapped in a `LayoutContext` that every constraint-aware provider participates in. Mutexed for the parallel walker so workers can negotiate against the same solver without racing.

- **Layout primitives** — `:::flex` (1D row/column with gap) and `:::grid` (2D, mixed `100px 1fr 1fr`-style track lists, independent `row-gap` / `column-gap`). Both share the same solver under the hood. Each cell still constrains its own bounds; the parent allocates the track.



The constraint substrate live: `:::flex` for 1D rows with cumulative-x positioning, `:::grid` for 2D with mixed `100px 1fr 1fr` tracks resolving through the kiwi solver, and `:::embedded-document` for recursive composition with parent-overlay attrs — all negotiating against the same `LayoutContext` under a mutex that lets the parallel walker fan out without racing.

~20,000 LOC of our own Zig + GLSL. ~300 unit tests. Three concurrent LLM streams, one live SVG generator, one live raster generator, a 60 Hz chart, plus 1D + 2D constraint-driven layout all in the same document, all ~7,600 fps Release.

What we refuse

- **HTML as the in-band format.** It is the wrong currency. If a renderer needs HTML, that is the renderer's problem, not the format's.
- **JavaScript hydration.** Live behaviour comes from native components registered with the host, not a runtime that costs 150 MB of memory and a JIT.

- **Closed presentation moats.** If the document only renders in one viewer, the document is hostage. Ours render the same source in this engine, in any markdown viewer (statically), and in a terminal (degraded gracefully). The richer behaviour layers on; the baseline always works.
 - **Hidden state.** The frontmatter is the state. The directives are the components. There is no compiled artifact, no bundler, no opaque blob. You can read what runs.
 - **Bandwidth-as-license.** A million tokens of context is not permission to be wasteful with the other 999,000. Tokens are reasoning budget. Format efficiency compounds at every layer.
-

Why this matters now

The wider AI ecosystem is at a fork. Down one path is HTML-first output, viewer lock-in, and presentation formats that only their authors can render. Down the other is markdown as the durable substrate — human-readable, agent-writable, version-controllable, viewer-agnostic — with rich behaviour layered on top.

We are betting on the second path because it is the only one that scales beyond a single company's UI. Markdown is the only format where a human in a terminal, an LLM in a context window, a code reviewer on GitHub, a small model on an edge device, and a native renderer on a GPU all meet on equal footing. Burning that ground to sell an artifact viewer is a category error.

The substrate we are building is the long-tail bet: that the right thing to do with markdown is *render it harder*, not *replace it*.

The horizon

The next sessions extend the substrate in directions the contract already anticipates. Two horizon items from the first writing of this manifesto have since landed — **persistent content-addressable cache** (session 10, `~/.cache/text_engine`) and **headless documents** (session 10, `headless=true` on any embed). What remains:

- **GPU-input → solver channel.** A compute shader writes a readback buffer per frame; the host wraps the result as `solver.suggestValue(var, x)`; surrounding layout reflows incrementally. The “fluid sim warps the document” demo. The whole constraint substrate was built nominally for this — now ready to claim.

- **Text intrusion.** `:::image {flow=around}` — markdown wraps around an SVG / raster figure via an `ExclusionShape` layered over the settled solver positions. Magazine-grade layout from a markdown file.
- **Measure-then-render protocol.** Lifting per-sibling negotiation (`flex-grow`, `justify=space-between`) into the solver itself, with every layout-aware element gaining a measure pass. Closes the loop on what `:::flex` and `:::grid` promise but currently approximate.
- **WASM-provenance components.** Drop-in sandboxed modules authored in any language that compiles to wasm32.
- **MCP-provenance components.** A component is an MCP server. The substrate hosts the bridge.
- **SPIR-V-provenance components.** Direct GPU code as a component. The renderer exposes raw Vulkan to the factory.
- **A terminal app** on top of the ANSI engine. The original motivating use case, demoted from chapter-one to chapter-N because the substrate became bigger than the terminal.
- **Dear-ImGui-tier game UI.** The Element contract was designed for this from session 2.

Every one of these slots into the existing Factory shape. The contract was built to hold them. That was the point.

In one line

Markdown is the universal interface. Make it live.

In the beginning was the markdown. And the markdown said: let there be live components. And there were live components. And it was good.

— somewhere in the *Encyclopaedia Galactica*, mid-2026

And the markdown spake further: let there be crisp zoom at every magnification, and emoji that flow inline, and ANSI fences inside the fences. And it was good.

— somewhere in the *Encyclopaedia Galactica*, late session 10

And the markdown spake further still: let there be a constraint solver beneath the page, and let `:::flex` walk its children in a row, and let `:::grid` arrange them in tracks both fixed and flexing, and let every cell negotiate its bounds against the same solver. And when the worker threads raced for the solver, let a mutex hold the line. And the document arranged itself, and resized, and held together. And it was good. And it was — improbably — still just markdown.

— somewhere in the Encyclopaedia Galactica, end of session 13



Appendix — Roadmap

The destination crystallised end-of-session-3 into a **live- document runtime**: markdown source as the declarative interface, native Vulkan components instantiated via `:::name {attrs}` block extensions, reactive frontmatter state, targeted LLM-streamed micro-updates. Full pitch + architectural mapping + design- decision rationale lives in `vision.md`.

This roadmap is the staging path from where we are (the full live-document substrate: parse → registry → cache → static interpolation → reactivity → input → first interactive component → `:::update` wire format → first streaming component → recursive document composition → constraint solver → flex + grid) to where we’re going (GPU-input channel, text intrusion, WASM / MCP / SPIR-V provenance, terminal app, game UI).

Shipped — block extension parser (stage 7a)

`markdown.zig` recognises `:::name {attrs}\nbody\n:::` syntax and emits a `custom` Element backed by a placeholder vtable. Approach: pre-scan source for `:::` blocks before cmark sees it; lift each block into a sidecar `Spec` slice; replace the byte range with a `<!-- te:N-->` HTML comment sentinel bracketed by blank lines so cmark treats it as a standalone `CMARK_NODE_HTML_BLOCK`; mapper intercepts the sentinel pattern and emits the `custom` Element. Keeps vendored cmark completely untouched.

Shipped — component registry + persistent cache (stage 7b)

Host registers `name → Factory`. Cache keyed by `Spec.id` or by auto-generated `auto:N`. `custom.ctx` carries the factory-produced instance pointer. `update` is the cache-hit path so an instance can react to attr changes between parses without being destroyed. GC sweeps every N parses to release stale entries.

Shipped — first concrete component (stage 7c)

`:::box` lives in `src/components/box.zig`. Reads `color` (named or `#RGB` / `#RRGGBB` / `#RRGGBBAA` hex), `width`, `height` (pixels or `%`), and `radius` (pixels) from `Spec.attrs`. Emits one rounded quad through the existing quad pipeline. `update` is wired so attr changes between parses mutate the cached instance in place — the “cheap edit” path the `:::update` micro-stream will eventually exploit. Missing-or-unparseable color → opaque magenta indicator.

Shipped — frontmatter state + static `${}` interpolation (stage 7d)

YAML frontmatter (hand-rolled subset — `key: value` pairs under a `state:` block; quoted/bare strings; comments honoured; nested maps + lists deferred). `${path}` interpolation in attribute values, resolved at component construction time.

Shipped — reactive state (stage 7e)

State is now observable. `state.set(path, value)` fires subscribers. The registry auto-subscribes a `Binding` callback per `${}` reference during `resolve`: when the path mutates, the binding re-substitutes the cached instance's templated attrs and calls `factory.update` with a fresh Spec. Demo cycles `box_color` through five colours every 1.5s. ~13.3k fps Release through the cycle, 99.6% cache hit rate.

Shipped — input handling (stage 7f)

`ElementVTable` grew an optional `on_input` callback. The walker appends an entry to `DrawList.hits` for any `custom` element that exposes `on_input` — a flat layer of (laid-out `Box`, `vtable`, `ctx`) tuples we walk in reverse on each mouse event so the deepest hit wins. First interactive component: `:::slider`. Drag updates a numeric state path declared via the `target` attribute.

Shipped — `:::update` micro-stream wire format (stage 8a)

The LLM-streamed-delta path — bypasses `cmark`, bypasses the `Element` walker, bypasses text layout. Two dispatch backends: component-target (`:::update {#id action=NAME}` → `Factory.handle_update`) and state-target (`:::update {target=state.path}` → reactive substrate propagation). Per-update overhead is one attr-list slice + one body dupe; demo holds the existing ~13.3k fps Release.

Shipped — `:::chart` streaming showcase (stage 8b)

The visceral component-target demo. Ring buffer of f32 samples, axis-aligned column rendering, `handle_update` accepts `action=append` / `action=clear`. The chart owns its data opaquely so it runs *without* a state `Binding`. 60 Hz synthetic data source (layered sines + noise) emits an update per tick. 12.0k fps Release with the full re-layout firing on every append.

Shipped — `:::embedded-document` (stage 9)

The composition flywheel kickoff. Built-in factory reads `src=` from disk, parses through `markdown.parseWithStateAndScope` with a fresh child `State` (`parent` pointer set so dirty bubbles up), grafts the resulting Element subtree into the host doc's layout. Non-`src` attrs overlay onto child state — parent always wins over child frontmatter. Scoped cache keys prevent ID collisions.

Shipped — headless documents (stage 10)

`:::embedded-document {headless=true}` parses the child markdown, populates child state, instantiates child components — but `layoutAndRender` short-circuits with a zero-size box and emits no draw data. Invisibility propagates behaviourally: nested visual content arbitrarily deep stays hidden regardless of its own `headless` flag, because the layout walker simply never descends. The cache-warming use case is the manifesto's pattern made concrete.

Shipped — async I/O channel (stage 12)

Two-layer concurrency primitive. `src/jobs.zig`: work-stealing thread pool (Chase-Lev deque) ported from valkyr. Workers own per-worker LIFO deque, steal FIFO when idle. `src/io_channel.zig`: `IoChannel` on top — `submitHttpGet` packages a request into a `Job`, worker runs the blocking fetch, pushes a `Completion` onto a mutex-guarded MPSC completion queue. Main thread calls `channel.drain(handler)` once per frame.

Shipped — live LLM authoring (stage 13a, 13a.5)

The headline payoff of stage 12. `IoChannel` grew a streaming variant; the new `:::llm-stream` component posts a `stream: true` chat request and renders the response as a child markdown tree that re-parses on every chunk. Tokens visibly stream into the document. The `provider=openai` mode covers DeepSeek, OpenRouter, OpenAI, Together, Groq, Mistral — all share the same `POST /chat/completions` SSE wire format. One implementation, an entire ecosystem.

Shipped — button, input field (stage 13b, 13c)

`:::button` — click → `registry.handleUpdate(target, action, body)`. `:::input` — single-line editable text field with full caret + UTF-8 buffer + arrows / home / end / backspace / delete / enter. Click to focus, Esc clears focus. Enter dispatches the buffer to a

target component. Demo: three input fields, one above each LLM stream — type a question, hit Enter, three providers answer in parallel.

Shipped — `:::svg` + `:::svg-stream` (stage 13d.1 – 13d.3)

End-to-end vector graphics. SVG parser (M/L/C/z paths with rgb fills) + CPU tessellator (recursive cubic-Bezier flattening + Mapbox-port earcut) + Vulkan triangle pipeline. Parallel tessellation gives 8.17× speedup on 125-path inputs. `:::svg-stream` targets Recraft V4.1 on OpenRouter — `stream:false`, parse SVG out of `choices[0].message.images[0].image_url.url`, tessellate, render. The Babel fish on page 5 was made this way.

Shipped — raster `:::image-stream` (stage 14c)

Mirror of `:::svg-stream` for image-class models (target: google/gemini-3.1-flash-image-preview). Same OpenAI-shaped wire format; data URL contains a PNG/JPG instead of an SVG, decoded via vendored `stb_image` into per-component `VkImage` + sampler. Render order: triangles → images → quads → glyphs.

Shipped — retained layout cache (stage 14a, 14f)

Per-block cache keyed on `(elem_id, max_w, theme)`. Content version held outside the key as an Entry field so a bump replaces the slot in place (no leak on 1000-chunk streams). Leaf-ish kinds (paragraph / heading / code_block) cache automatically; custom components opt in via `vtable.content_version`. Idle: 97.9% cache hit rate, 56 entries, ~7600 fps with chart at 60 Hz + box color cycle. Cost-aware parallel-walk classification (14f) keeps chart-only-dirty frames serial.

Shipped — persistent asset cache (stage 14e)

Browser-style content-addressable cache for expensive remote assets. Configurable byte budget (default 500 MB), LRU eviction on overflow, atomic manifest writes. Each cacheable consumer derives its own key as a hash of provider + endpoint + model + prompt + max_tokens, prefixed by a schema version. Cache hit goes straight to the parser; no IoChannel traffic, no spinner, no charge.

Shipped — persistent reactive state (stage 13b.2)

`State.saveToFile` / `loadFromFile`. JSON format, atomic write. New `persist_dirty` flag independent of render `dirty`; host throttles disk writes on its own cadence. Throttled flush every 60 frames (~1 s at 60 fps) if dirty. The slider-drag pattern (60 sets/s) coalesces to one disk write per second. Restart picks up where the user left off.

Shipped — kiwi constraint solver (stage 15A)

Pure-Zig port of Chris Colbert's kiwi (modern Cassowary, BSD-3, ~3000 LOC C++). Incremental dual-simplex tableau, four-tier strengths (required / strong / medium / weak), edit variables for reactive inputs. Lives at `src/layout/kiwi/` as a self-contained module with zero `text_engine` deps — clean boundary, tested in isolation. ~3,000 LOC + 300+ unit tests (Zig translation of the canonical kiwi C++ test corpus).

Shipped — LayoutContext + `:::box` integration (stage 15B)

`LayoutContext` wraps the kiwi `Solver` plus a `bounds_map` keyed by `@intFromPtr(component_ctx)` — opaque stable keys, no string ids required from components. Each frame: `solver.reset()`, constraint-aware providers declare constraints on their children, `updateVariables` settles, walker reads positions. `:::box` migrated to the constraint path: declares `x`, `y`, `width`, `height` variables, posts the box-model edit constraints, reads its laid-out rect back through `getBounds`.

Shipped — `:::flex` provider (stage 15C)

First constraint-aware provider. `:::flex {#id direction=row gap=N}` arranges children left-to-right (column direction parked) with a uniform gap. Body re-parsed via `markdown.parseWithStateAndScope` so children carry full reactive state. Flex-grow / shrink / justify=space-between deferred — those need a measure-pass protocol (parked).

Shipped — `:::grid` provider + parallel-walk hardening (stage 15F.1, 15d.1, 15e)

The first multi-axis layout primitive. `:::grid {#id columns="100px 1fr 1fr" column-gap=N row-gap=N}` arranges children row-major across mixed fixed/flex tracks. Track parser handles `100px`, `100`, `1fr`, `2.5fr`. Resolver sums fixed widths, subtracts from `(avail - total_gap)`, distributes the remainder by `fr` weight.

Mid-session-13 a parallel-walk race surfaced: the stage-14b dispatcher reached `box.layoutViaConstraints` on worker threads, multiple workers wrote to the shared `LayoutContext.solver` concurrently, and Zig 0.14's `AutoArrayHashMap` pointer-stability safety lock tripped. Fix: `LayoutContext` grew a `std.Thread.Mutex` held across the full add-constraints + `updateVariables` + value-read sequence, ~19 LOC of real change. Critical section ~10-30µs per box. Re-enabled the parallel walker for grid + flex layouts under load.

Next — GPU-input channel + text intrusion (stage 15D, 15E)

The two remaining phases of the stage 15 substrate plan, both of which the kiwi port + `LayoutContext` + parallel-walk hardening were built nominally to host.

- **GPU-input → solver channel (Phase D).** Compute shader writes a readback buffer per frame; host wraps the result as `solver.suggestValue(var, x)`; surrounding layout reflows incrementally. The “fluid sim warps the document” demo. The constraint substrate is the load-bearing piece; the rest is plumbing the GPU readback channel onto solver edit variables.
- **Text intrusion / exclusion (Phase E).** `:::image {flow=around}` — markdown wraps around an SVG / raster figure via an `ExclusionShape` layered over the settled solver positions. CSS `shape-outside` semantics; per-line break decisions consult the exclusion shapes. Magazine-grade layout from a markdown file.

Strength discipline (required / strong / medium / weak) and the measure-then-render protocol (lifting per-sibling negotiation — `flex-grow`, `justify=space-between` — into the solver itself, parked from stage 15C) layer onto either phase when each one lands.

Eventually — LM connection (tier 3)

Once the rendering channels and the live-document runtime are both stable, plug in real LM signals as the producer of state mutations + `:::update` streams. The LM doesn't just produce text — it produces a *live document* that updates in place as the model's understanding evolves.

Parked

Captured here, surface when relevant:

- **Atlas overflow** — `error.AtlasFull` after enough unique glyphs. LRU eviction or grow + recreate descriptor.
- **Gamma correction** — fine at 20 px, matters at 14 px.

- **Bidirectional text** — HB shapes each run correctly, but line composition assumes LTR.
- **Break-anywhere wrap** — a single word wider than `max_w` overflows. Character-level break fallback when no whitespace is available.
- **Naming.** `text_engine` is visibly the wrong name for what's becoming a live-document runtime. Rename when the runtime layer above the contract is anchored. Candidates: `glow`, `forge`, `litho`, or something tied to matryoshka.